- **Novel applications** (Sec. 6): We describe novel applications powered by Milvus. In particular, we build a series of **10** applications[2] on top of Milvus to demonstrate its broad applicability including image search, video search, chemical structure analysis, COVID-19 dataset search, personalized recommendation, biological multi-factor authentication, intelligent question answering, image-text retrieval, cross-modal pedestrian search, and recipe-food search.

2 SYSTEM DESIGN

In this section, we present an overview of Milvus. Figure 1 shows the architecture of Milvus with three major components: query engine, GPU engine, and storage engine. The query engine supports efficient query processing over vector data and it is optimized for modern CPUs by reducing cache misses and leveraging SIMD instructions. The GPU engine is a co-processing engine that accelerates performance with vast parallelism. It also supports multiple GPU devices for efficiency. The storage engine enables data durability and incorporates an LSM-based structure for dynamic data management. It runs on various file systems (including local file systems, Amazon S3, and HDFS) with a bufferpool in memory.

2.1 Query Processing

We first present the concept of entity used in Milvus and then explain query types, similarity functions, and application interfaces.

Entity. To best capture versatile data science and AI applications, Milvus supports query processing over both vector data and non-vector data. We define the term *entity* as follows to incorporate the two. Each entity in Milvus is described as one or more vectors and optionally some numerical attributes (non-vector data). For example, in the image search application, the numerical attributes can represent the age and height of a person in addition to possibly multiple machine-learned feature vectors of his/her photos (e.g., describing front-face, side-face, or posture [10]). In the current version of Milvus, we only support numerical attributes as observed from many applications. But in the future, we plan to support categorical attributes with indexes like inverted lists or bitmaps [64].

Query types. Milvus supports three primitive query types:

- Vector query: This query type is the traditional vector similarity search [33, 41, 48, 49], where each entity is described as a single vector. The system returns k most similar vectors where k is a user-input parameter.
- Attribute filtering: Each entity is specified by a single vector and some attributes [65]. The system returns k most similar vectors while adhering to the attributes constraints. As an example in recommender systems, users want to find similar clothes to a given query image while the price is below $100.
- Multi-vector query: Each entity is stored as multiple vectors [10]. The query returns top-k similar entities according to an aggregation function (e.g., weighted sum) between multiple vectors.

Similarity functions. Milvus offers commonly used similarity metrics, including Euclidean distance, inner product, cosine similarity, Hamming distance, and Jaccard distance, allowing applications to explore vector similarity in the most effective approach.

[2]https://github.com/milvus-io/bootcamp/tree/master/EN_solutions

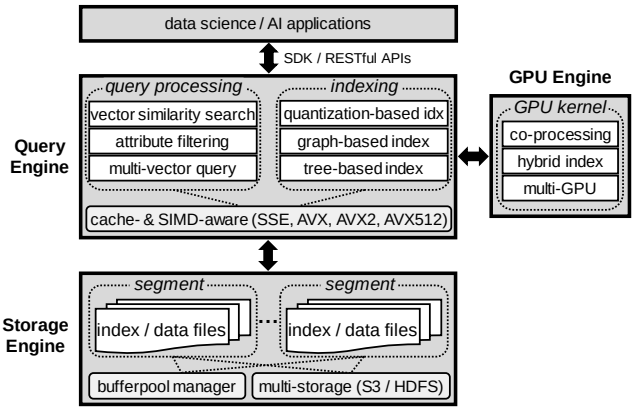

Figure 1: System architecture of Milvus

Application interfaces. Milvus provides easy-to-use SDK (software development kit) interfaces that can be directly called in applications written in various languages including Python, Java, Go, and C++. Milvus also supports RESTful APIs for web applications.

2.2 Indexing

Indexing is of tremendous importance to query processing in Milvus. But a challenging issue we face is to decide which indexes to support in Milvus, because there are numerous indexes developed for vector similarity search. The latest benchmark [41] shows that there is no winner in all scenarios and each index comes with tradeoffs in performance, accuracy, and space overhead.

In Milvus, we mainly support two types of indexes:[3] quantization-based indexes (including IVF_FLAT [3, 33, 35], IVF_SQ8 [3, 35], and IVF_PQ [3, 22, 33, 35]) and graph-based indexes (including HNSW [49] and RNSG [20]) to serve different applications. The design decision is based on factors including the latest literature review [41], industrial-strength systems (e.g., Alibaba PASE [68], Alibaba AnalyticDB-V [65], Jingdong Vearch [39]), open-source libraries (e.g., Facebook Faiss [3, 35]), and inputs from customers. We exclude LSH-based approaches because they have lower accuracy than quantization-based approaches on billion-scale data [65, 68].

Considering there are many new indexes coming out every year, Milvus is designed to easily incorporate the new indexes with a high-level abstraction. Developers only need to implement a few pre-defined interfaces for adding a new index. Our hope is that Milvus can eventually become a standard platform for vector data management with versatile indexes.

2.3 Dynamic Data Management

Milvus supports efficient insertions and deletions by adopting the idea of LSM-tree [47]. Newly inserted entities are stored in memory first as MemTable. Once the accumulated size reaches a threshold, or once every second, the MemTable becomes immutable and then gets flushed to disk as a new segment. Smaller segments are merged into larger ones for fast sequential access. Milvus implements a tiered merge policy (also used in Apache Lucene) that aims to merge segments of approximately equal sizes until a configurable size limit (e.g., 1GB) is reached. Deletions are supported in the same out-of-place approach except that the obsoleted vectors are

[3]Milvus also supports tree-based indexes, e.g., ANNOY [1].

removed during segment merge. Updates are supported by deletions and insertions. By default, Milvus builds indexes only for large segments (e.g., > 1GB) but users are allowed to manually build indexes for segments of any size if necessary. Both index and data are stored in the same segment. Thus, the segment is the basic unit of searching, scheduling, and buffering.

Milvus offers snapshot isolation to make sure reads and writes share a consistent view and do not interfere with each other. We present the details of snapshot isolation in Sec. 5.2.

2.4 Storage Management

As mentioned in Sec. 2.1, each entity is expressed as one or more vectors and optionally some attributes. Thus, each entity can be regarded as a row in an entity table. To facilitate query processing, Milvus physically stores the entity table in a columnar fashion.

Vector storage. For single-vector entities, Milvus stores all the vectors continuously without explicitly storing the row IDs. In this way, all the vectors are sorted by row IDs. Given a row ID, Milvus can directly access the corresponding vector since each vector is of the same length. For multi-vector entities, Milvus stores the vectors of different entities in a columnar fashion. For example, assuming that there are three entities (A, B, and C) in the database and each entity has two vectors $\mathbf{v}_1$ and $\mathbf{v}_2$, then all the $\mathbf{v}_1$ of different entities are stored together and all the $\mathbf{v}_2$ are stored together. That is, the storage format is $\{A.\mathbf{v}_1, B.\mathbf{v}_1, C.\mathbf{v}_1, A.\mathbf{v}_2, B.\mathbf{v}_2, C.\mathbf{v}_2\}$.

Attribute storage. The attributes are stored column by column. In particular, each attribute column is stored as an array of $\langle key, value \rangle$ pairs where the *key* is the attribute value and *value* is the row ID, sorted by the *key*. Besides that, we build skip pointers (i.e., min/max values) following Snowflake [16] as indexing for the data pages on disk. This allows efficient point query and range query in that column, e.g., price is less than $100.

Bufferpool. Milvus assumes that most (if not all) data and index are resident in memory for high performance. If not, it relies on an LRU-based buffer manager. In particular, the caching unit is a segment, which is the basic searching unit as explained in Sec. 2.3.

Multi-storage. For flexibility and reliability, Milvus supports multiple file systems including local file systems, Amazon S3, and HDFS for the underlying data storage. This also facilitates the deployment of Milvus in the cloud.

2.5 Heterogeneous Computing

Milvus is highly optimized for the heterogeneous computing platform that includes CPUs and GPUs. Sec. 3 presents the details.

2.6 Distributed System

Milvus can function as a distributed system deployed across multiple nodes. It adopts modern design practices in distributed systems and cloud systems such as storage/compute separation, shared storage, read/write separation, and single-writer-multi-reader. Sec. 5.3 explains more.

3 HETEROGENEOUS COMPUTING

In this section, we present the optimizations for Milvus to best leverage the heterogeneous computing platform involving both CPUs and GPUs to achieve high performance.

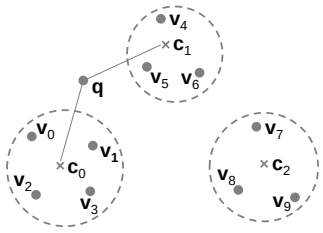

Figure 2: An example of quantization

As explained in Sec. 2.2, Milvus mainly supports quantization-based indexes (including IVF_FLAT [3, 33, 35], IVF_SQ8 [3, 35], and IVF_PQ [3, 22, 33, 35]) and graph-based indexes (including HNSW [49] and RNSG [20]). In this section, we use quantization-based indexes to illustrate our optimizations because they consume much less memory and are much faster to build index while achieving decent query performance when compared to graph-based indexes [65, 68]. Note that many optimizations (such as SIMD and GPU optimizations) can be applied to graph-based indexes.

3.1 Background

Before diving into optimizations, we explain vector quantization and quantization-based indexes. The main idea of vector quantization is to apply a quantizer z to map a vector $\mathbf{v}$ to a codeword $z(\mathbf{v})$ chosen from a codebook C [33]. The K-means clustering algorithm is commonly used to construct the codebook C where each codeword is the centroid and $z(\mathbf{v})$ is the closest centroid to $\mathbf{v}$. Figure 2 shows an example of 10 vectors ($\mathbf{v}_0$ to $\mathbf{v}_9$) of three clusters with centroids being $\mathbf{c}_0$ to $\mathbf{c}_2$, then $z(\mathbf{v}_0)$, $z(\mathbf{v}_1)$, $z(\mathbf{v}_2)$, or $z(\mathbf{v}_3)$ is $\mathbf{c}_0$.

Quantization-based indexes (such as IVF_FLAT [3, 33, 35], IVF_SQ8 [3, 35], and IVF_PQ [3, 22, 33, 35]) use two quantizers: coarse quantizer and fine quantizer. The coarse quantizer applies the K-means algorithm (e.g., K is 16384 in Milvus and Faiss [3]) to cluster vectors into K buckets. And the fine quantizer encodes the vectors within each bucket. Different indexes may use different fine quantizers. IVF_FLAT uses the original vector representation; IVF_SQ8 uses a compressed representation for the vectors by adopting one-dimensional quantizer (called "scalar quantizer") to compress a 4-byte float value to a 1-byte integer; and IVF_PQ uses product quantization that splits each vector into multiple sub-vectors and applies K-means for each sub-space.

Query processing (of a query $\mathbf{q}$) over quantization-based indexes takes two steps: (1) Find the closest n_{probe} buckets (or clusters) based on the distance between $\mathbf{q}$ and the centroid of each bucket. For example, assuming n_{probe} is 2 in Figure 2, then the closest two buckets of $\mathbf{q}$ are centered at $\mathbf{c}_0$ and $\mathbf{c}_1$. The parameter n_{probe} controls the tradeoff between accuracy and performance. Higher n_{probe} produces better accuracy but worse performance. (2) Search within each of the n_{probe} relevant buckets based on different fine quantizers. For example, if the index in Figure 2 is IVF_FLAT, then it needs to scan the vectors $\mathbf{v}_0$ to $\mathbf{v}_6$ in the two buckets.

3.2 CPU-oriented Optimizations

3.2.1 Cache-aware Optimizations in Milvus

The fundamental problem for query processing over quantization-based indexes is that, given a collection of m queries $\{\mathbf{q}_1, \mathbf{q}_2, ..., \mathbf{q}_m\}$ and a collection of n data vectors $\{\mathbf{v}_1, \mathbf{v}_2, ..., \mathbf{v}_n\}$, how to quickly